

Homework 4

Daniel Oluwafemi Olofin

September 08, 2025

1. This problem will have you create, modify, and expand a data frame.

a. Create a small data frame describing a set of objects like I did with cardens. There should be at least four observations, and at least three variables, with at least one variable that is numeric and at least one that is a factor. The first column should be something usable as a unique identifier.

```
# Create a data frame
fruit_df <- data.frame(
  ID = c("F001", "F002", "F003", "F004"), # Unique identifier
  Name = factor(c("Apple", "Banana", "Cherry", "Date")), # Factor variable
  Price = c(1.2, 0.5, 2.5, 3.0) # Numeric variable
)

# View the data frame
print(fruit_df)
```

```
##      ID   Name Price
## 1 F001  Apple   1.2
## 2 F002 Banana   0.5
## 3 F003 Cherry   2.5
## 4 F004   Date   3.0
```

b. Use logical indexing on the identifier column to select a row, and use a column/variable name to select a column, and use these to update one of the values in the data frame.

```
# Update the price of Banana using logical indexing
fruit_df[fruit_df$ID == "F002", "Price"] <- 0.6

# Check the updated data frame
print(fruit_df)
```

```
##      ID   Name Price
## 1 F001  Apple   1.2
## 2 F002 Banana   0.6
## 3 F003 Cherry   2.5
## 4 F004   Date   3.0
```

```
## 1 F001 Apple 1.2
## 2 F002 Banana 0.6
## 3 F003 Cherry 2.5
## 4 F004 Date 3.0
```

c. Use `rbind()` to add a new observation, and `cbind()` to add a new variable

```
# Create a new row as a data frame
new_fruit <- data.frame(ID = "F005", Name = factor("Elderberry"), Price = 4.0)

# Define a new column (factor variable)
fruit_colors <- c("Red", "Yellow", "Red", "Brown", "Purple")

# Use rbind() to add the new observation
fruit_df <- rbind(fruit_df, new_fruit)

# Use cbind() to add the new column
fruit_df <- cbind(fruit_df, Color = factor(fruit_colors))

# View final data frame
print(fruit_df)
```

```
##      ID      Name Price Color
## 1 F001 Apple 1.2 Red
## 2 F002 Banana 0.6 Yellow
## 3 F003 Cherry 2.5 Red
## 4 F004 Date 3.0 Brown
## 5 F005 Elderberry 4.0 Purple
```

2. The code below will generate a small random matrix. Write code that will identify and remove the row with the smallest sum of absolute values.

```
set.seed(123)
x <- matrix(sample(-5:5, 15, replace = TRUE), nrow = 5)

# Print the original matrix
print(x)
```

```
##      [,1] [,2] [,3]
## [1,] -3 5 4
## [2,] -3 -1 5
## [3,] 4 -2 -1
## [4,] -4 0 -3
## [5,] 0 3 5
```

```

# Compute the sum of absolute values for each row
row_sums <- rowSums(abs(x))

# Identify the index of the row with the smallest sum
min_row_index <- which.min(row_sums)

# Remove the row with the smallest sum
x <- x[-min_row_index, , drop = FALSE]

# Print the updated matrix
print(x)

```

```

##      [,1] [,2] [,3]
## [1,]  -3   5   4
## [2,]  -3  -1   5
## [3,]  -4   0  -3
## [4,]   0   3   5

```

3. Identify and display the most frequent entry in the vector `x`. If there are ties, returning one of the values is sufficient. Hint: The `table()` function with argument `decreasing = TRUE` makes this much shorter than it might be otherwise.

```

# Generate a random sample
x <- sample(1:5, 20, replace = TRUE)

# Print the vector
print(x)

```

```

## [1] 1 4 1 1 5 3 2 2 1 3 4 1 3 5 4 2 5 1 1 2

```

```

# Create a frequency table, sorted in decreasing order
freq_table <- sort(table(x), decreasing = TRUE)

# Extract and display the most frequent value
most_frequent <- as.numeric(names(freq_table)[1])

print(most_frequent)

```

```

## [1] 1

```

4. Highlight the following line and run it several times. Sometimes it returns a value, and sometimes it returns NULL. Explain why sometimes NULL is returned. Without changing the randomization aspect, find a way to ensure it never returns NULL.

```
rownames(mtcars[ ,  
sample(1:4, sample(1:4, size = 1))  
])[[1]]
```

```
## [1] "Mazda RX4"
```

- **why is it null:** It returns NULL when no columns are selected from `mtcars`, which happens if `sample(1:4, size = 1)` randomly chooses 0. In this case, `mtcars[ , selected_columns]` results in an empty data frame, and `rownames()` applied to an empty data frame returns an empty vector. Attempting to access `[[1]]` on an empty vector leads to NULL.

solving it not to return NULL

```
rownames(mtcars[ , sample(1:4, sample(1:4, size = 1)), drop = FALSE])[[1]]
```

```
## [1] "Mazda RX4"
```

5. Write a line of code that will remove any values from `x` that are not repeated.

Hint: My solution used `%in%`, `as.numeric`, `names`, `table`, `>`.

```
x <- sample(c(-10:10), sample(20:40, 1), replace = TRUE)  
  
# Remove values that appear only once  
x <- x[x %in% as.numeric(names(table(x)[table(x) > 1]))]  
  
# Print the updated x  
print(x)
```

```
## [1] -9 -3 2 7 -5 4 4 5 -5 -3 5 6 7 6 -9 2
```